> EXPLAIN ANALYZE

SQL Query Optimization in MySQL

Comparing query plans and execution times on the *employees* sample database (≈300K employees, ≈2.8M salary records).

`MySQL 8` `InnoDB` `EXPLAIN ANALYZE` `DBeaver` `Indexes` `JOINs`

Summary

#	Topic	Key finding
1	Select only needed columns	Same plan and cost; B ≈27% faster in one run (likely cache)
2	Indexes	Full scan of 300K rows to find 224 matches; indexed run needs local DB
3	UNION vs UNION ALL	Cost 76,590 vs 60,411; identical result (251 rows)
4	Subquery vs JOIN	MySQL 8 rewrites IN into a semijoin; identical plan cost
5	JOIN optimization	Fewer rows ≠ faster: 2,080 ms vs 1,850 ms; filter variant 1,334 ms

Method: each query was analyzed with `EXPLAIN ANALYZE` on the course MySQL server. Times are from single runs unless noted, so small differences (a few ms) fall within measurement noise; compare plans and cost, not just milliseconds.

01 Select Only the Columns You Need

Task. Find employees hired after 2000. (A) select all columns; (B) select only `emp_no` and `hire_date`.

A `SELECT *`

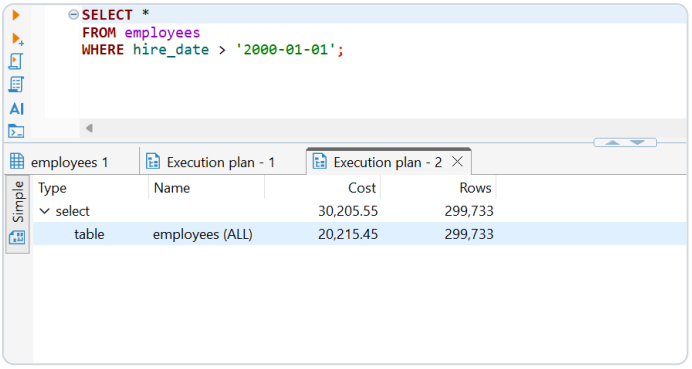
```
EXPLAIN ANALYZE
SELECT *
FROM employees
WHERE hire_date > '2000-01-01';
```

```
-> Filter: (employees.hire_date > DATE'2000-01-01')  (cost=30206 rows=99901) (actual time=15..108 rows=12 loops=1)
    -> Table scan on employees  (cost=30206 rows=299733) (actual time=0.189..89.9 rows=300024 loops=1)
```

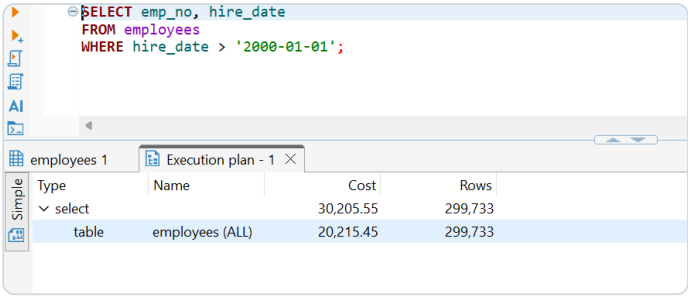
B Only `emp_no` , `hire_date`

```
EXPLAIN ANALYZE
SELECT emp_no, hire_date
FROM employees
WHERE hire_date > '2000-01-01';
```

```
-> Filter: (employees.hire_date > DATE'2000-01-01')  (cost=30206 rows=99901) (actual time=9.82..78.6 rows=12 loops=1)
    -> Table scan on employees  (cost=30206 rows=299733) (actual time=0.24..59.2 rows=300024 loops=1)
```



DBeaver execution plan, variant A



DBeaver execution plan, variant B

	A: SELECT *	B: 2 columns
Access method	Full table scan	Full table scan
Cost	30206	30206
Rows scanned	300,024	300,024
Rows returned	12	12
Actual time	≈108 ms	≈79 ms

Conclusion. Both queries use the same plan: a full table scan, because `hire_date` has no index. The cost is identical. B was faster in this single run (≈79 ms vs ≈108 ms), but since the plan and the number of rows read are the same and the result is tiny (12 rows), much of the gap is probably caching and measurement noise. Selecting fewer columns reduces the amount of data built, transferred and rendered by the client, so the effect grows with large result sets and wide rows. It also enables covering indexes.

02 Using Indexes Correctly

> **Task.** Return `emp_no` of employees named "Mary". (A) without an index on `first_name`; (B) after creating one.

A No index

```
EXPLAIN ANALYZE
SELECT emp_no
FROM employees
WHERE first_name = 'Mary';
```

```
-> Filter: (employees.first_name = 'Mary')  (cost=30206 rows=29973) (actual time=0.286..82 rows=224 loops=1)
    -> Table scan on employees  (cost=30206 rows=299733) (actual time=0.279..60.8 rows=300024 loops=1)
```

B With an index

```
CREATE INDEX idx_first_name ON employees (first_name);

EXPLAIN ANALYZE
SELECT emp_no
FROM employees
WHERE first_name = 'Mary';
```

Not measured. The shared course server rejected `CREATE INDEX` (`SQL Error 1142: INDEX command denied to user 'student'`), so variant B was not executed. The expected behavior below is based on how InnoDB works, not on a measurement.

	A: measured	B: expected (not measured)
Access method	Full table scan + filter	Covering index lookup (`idx_first_name`)
Rows examined	300,024	≈ the 224 matching rows
Time	≈82 ms	a small fraction of A

Conclusion. Without an index, MySQL must read all 300,024 rows to find 224 matches. A secondary InnoDB index stores `(first_name, emp_no)`, so after indexing, this query can be answered from the index alone (a covering index) with no table access. Indexes are not free: they use disk space and slow down `INSERT` / `UPDATE` / `DELETE`, so index only the columns you filter or join on.

03 UNION vs UNION ALL

> **Task.** Combine employees hired in January 1999 with those hired in January 2000.

A `UNION`

```sql
EXPLAIN ANALYZE
SELECT emp_no, first_name, last_name, hire_date
FROM employees
WHERE hire_date >= '1999-01-01' AND hire_date < '1999-02-01'
UNION
SELECT emp_no, first_name, last_name, hire_date
FROM employees
WHERE hire_date >= '2000-01-01' AND hire_date < '2000-02-01';
```

```
-> Table scan on <union temporary>  (cost=75755..76590 rows=66594) (actual time=194..194 rows=251 loops=1)
    -> Union materialize with deduplication  (cost=75755..75755 rows=66594) (actual time=194..194 rows=251 loops=1)
        -> Filter: (hire_date >= '1999-01-01' and hire_date < '1999-02-01')  (actual time=0.538..98.2 rows=238)
            -> Table scan on employees  (cost=30206 rows=299733) (actual time=0.197..78.5 rows=300024)
        -> Filter: (hire_date >= '2000-01-01' and hire_date < '2000-02-01')  (actual time=13.3..95.5 rows=13)
            -> Table scan on employees  (cost=30206 rows=299733) (actual time=0.192..76.2 rows=300024)
```

B `UNION ALL`

```sql
EXPLAIN ANALYZE
SELECT emp_no, first_name, last_name, hire_date
FROM employees
WHERE hire_date >= '1999-01-01' AND hire_date < '1999-02-01'
UNION ALL
SELECT emp_no, first_name, last_name, hire_date
FROM employees
WHERE hire_date >= '2000-01-01' AND hire_date < '2000-02-01';
```

```
-> Append  (cost=60411 rows=66594) (actual time=0.526..201 rows=251 loops=1)
    -> Stream results  (cost=30206 rows=33297) (actual time=0.525..101 rows=238 loops=1)
        -> Filter: (hire_date >= '1999-01-01' and hire_date < '1999-02-01')  (actual time=0.517..101 rows=238)
            -> Table scan on employees  (cost=30206 rows=299733) (actual time=0.181..80.4 rows=300024)
    -> Stream results  (cost=30206 rows=33297) (actual time=13.1..100 rows=13 loops=1)
        -> Filter: (hire_date >= '2000-01-01' and hire_date < '2000-02-01')  (actual time=13..100 rows=13)
            -> Table scan on employees  (cost=30206 rows=299733) (actual time=0.2..79.8 rows=300024)
```

	A: UNION	B: UNION ALL
Top node	Materialize with deduplication	Append (stream)
Estimated cost	76,590	60,411 (≈21% lower)
Rows returned	251	251
Actual time	≈194 ms	≈201 ms

Conclusion. Both return 251 rows (238 + 13): the date ranges don't overlap, so there are no duplicates and the deduplication in `UNION` removed nothing. The optimizer correctly estimates `UNION` as more expensive (a temporary table plus dedup), but actual time is dominated by the two full table scans (~80 ms each), so the measured difference is within noise. Use `UNION` only when duplicates are possible and must be removed; use `UNION ALL` when duplicates are impossible or wanted.

04 Avoid Subqueries When Possible

> **Task.** Find employees who are also managers. (A) with an `IN` subquery; (B) with a `JOIN`.

A Subquery

```
EXPLAIN ANALYZE
SELECT e.emp_no, e.first_name, e.last_name
FROM employees e
WHERE e.emp_no IN (SELECT dm.emp_no FROM dept_manager dm);
```

```
-> Nested loop inner join (LooseScan)  (cost=11.1 rows=24) (actual time=0.0544..0.0943 rows=24 loops=1)
    -> Remove duplicates from input sorted on PRIMARY  (cost=2.68 rows=24) (actual time=0.034..0.0421 rows=24)
        -> Covering index scan on dm using PRIMARY  (cost=2.68 rows=24) (actual time=0.0323..0.0378 rows=24)
    -> Single-row index lookup on e using PRIMARY (emp_no = dm.emp_no)  (cost=6.1 rows=1) (actual time=0.00186..0.00189 rows=1
loops=24)
```

B JOIN

```
EXPLAIN ANALYZE
SELECT e.emp_no, e.first_name, e.last_name
FROM employees e
JOIN dept_manager dm ON e.emp_no = dm.emp_no;
```

```
-> Nested loop inner join  (cost=11.1 rows=24) (actual time=0.0534..0.111 rows=24 loops=1)
    -> Covering index scan on dm using dept_no  (cost=2.65 rows=24) (actual time=0.0315..0.0398 rows=24)
    -> Single-row index lookup on e using PRIMARY (emp_no = dm.emp_no)  (cost=0.254 rows=1) (actual time=0.00261..0.00265 rows=1
loops=24)
```

	A: IN subquery	B: JOIN
Strategy	Semijoin (LooseScan)	Nested loop inner join
Cost	11.1	11.1
Rows returned	24	24
Actual time	≈0.094 ms	≈0.111 ms

Conclusion. MySQL 8 rewrote the `IN` subquery into a semijoin (LooseScan), so plan A is essentially the JOIN from B: 24 rows from `dept_manager` and 24 primary-key lookups in `employees`. Here the subquery adds no overhead. In harder cases (correlated subqueries, `NOT IN`, aggregates, older MySQL versions) a subquery can run once per outer row or go through a temporary table, so a JOIN is the more predictable choice.

05 Optimizing JOIN Operations

> **Task.** Compare a simple JOIN (all salaries) with a more complex JOIN that returns only the latest salary per employee.

A Simple JOIN: all salaries

```
EXPLAIN ANALYZE
SELECT e.emp_no, e.first_name, e.last_name, s.salary, s.from_date
FROM employees e
JOIN salaries s ON e.emp_no = s.emp_no;
```

```
-> Nested loop inner join  (cost=387443 rows=2.82e+6) (actual time=0.313..1850 rows=2.84e+6 loops=1)
    -> Table scan on e  (cost=30206 rows=299733) (actual time=0.287..97.1 rows=300024 loops=1)
    -> Index lookup on s using PRIMARY (emp_no = e.emp_no)  (cost=0.252 rows=9.4) (actual time=0.00329..0.00494 rows=9.48 loops=300024)
```

B Latest salary per employee (`MAX(from_date)`)

`GROUP BY` alone cannot guarantee the row with the latest `from_date`, so the latest date is computed in a derived table and joined back.

```
EXPLAIN ANALYZE
SELECT e.emp_no, e.first_name, e.last_name, s.salary, s.from_date
FROM employees e
JOIN salaries s ON e.emp_no = s.emp_no
JOIN (
  SELECT emp_no, MAX(from_date) AS max_from
  FROM salaries
  GROUP BY emp_no
) last_s ON s.emp_no = last_s.emp_no AND s.from_date = last_s.max_from;
```

```
-> Nested loop inner join  (cost=245440 rows=302077) (actual time=925..2080 rows=300024 loops=1)
    -> Nested loop inner join  (cost=139713 rows=302077) (actual time=925..1484 rows=300024 loops=1)
        -> Filter: (last_s.max_from is not null)  (cost=33986 rows=302077) (actual time=925..991 rows=300024)
            -> Table scan on last_s  (cost=176541..180319 rows=302077) (actual time=925..966 rows=300024)
                -> Materialize  (cost=176541..176541 rows=302077) (actual time=925..925 rows=300024)
                    -> Covering index skip scan for grouping on salaries using PRIMARY  (cost=106938) (actual time=0.0511..848)
        -> Single-row index lookup on e using PRIMARY (emp_no = last_s.emp_no)  (cost=0.25) (actual time=0.00142..0.00146 loops=300024)
    -> Single-row index lookup on s using PRIMARY (emp_no, from_date)  (cost=0.25) (actual time=0.00176..0.00179 loops=300024)
```

C Optimization: filter on current salary

```
EXPLAIN ANALYZE
SELECT e.emp_no, e.first_name, e.last_name, s.salary, s.from_date
FROM employees e
JOIN salaries s ON e.emp_no = s.emp_no
WHERE s.to_date = '9999-01-01';
```

```
-> Nested loop inner join  (cost=384717 rows=283843) (actual time=1.39..1334 rows=240124 loops=1)
    -> Filter: (s.to_date = DATE'9999-01-01')  (cost=285373 rows=283843) (actual time=1.37..876 rows=240124 loops=1)
        -> Table scan on s  (cost=285373 rows=2.84e+6) (actual time=1.36..668 rows=2.84e+6 loops=1)
    -> Single-row index lookup on e using PRIMARY (emp_no = s.emp_no)  (cost=0.25 rows=1) (actual time=0.00166..0.0017 rows=1
loops=240124)
```

	A: all salaries	B: latest (MAX)	C: to_date filter
Rows returned	2,840,000	300,024	240,124
Cost	387,443	245,440	384,717
Actual time	≈1,850 ms	≈2,080 ms	≈1,334 ms
Reads of `salaries`	1 (PK lookups)	2 (skip scan + PK lookups)	1 (full scan)
Extra work	none	aggregation + materialization	filter



Also tried. A `ROW_NUMBER() OVER (PARTITION BY emp_no ORDER BY from_date DESC)` variant failed on the shared server with `SQL Error 1114: The table is full`, because the window function has to materialize all 2.8M joined rows in a temporary table.

Conclusion. B returns about 9× fewer rows than A but is not faster (≈2,080 ms vs ≈1,850 ms): the optimizer must aggregate, materialize a 300K-row derived table and do two sets of 300K primary-key lookups, so plan complexity cancels out the smaller result. C is the fastest (≈1,334 ms) since it needs no aggregation or temporary table, but it is **not equivalent** to B: it returns only employees with a currently active salary (240,124 vs 300,024). It still scans all 2.84M salary rows; an index on `salaries(to_date)` or `(to_date, emp_no)` would remove that scan. Takeaway: judge a query by plan complexity, not only by the number of rows it returns.